DOCUMENTACIÓN TÉCNICA

Sistema de Detección de Fatiga en Conductores Mediante Visión por Computadora e Inteligencia Artificial

FastAPI TensorFlow / Keras MobileNetV2 OpenCV Docker

Alumno	Angel de Jesús Canela Figueroa
Materia	Visión Por Computadora Y Machine Learning
Institución	MachineLearningWEB 2026
Metodología	CRISP-DM
Fecha	Mayo 2026
Plataforma	Mac M3 (Apple Silicon) — macOS

Índice de Contenidos

#	Sección
1.	Descripción del Proyecto
2.	Estructura del Proyecto
3.	Herramientas y Versiones
4.	Dataset y Preparación de Datos
5.	Arquitectura del Modelo de IA
6.	Entrenamiento y Resultados
7.	Pipeline de Preprocesamiento en Producción
8.	Proceso de Implementación — Paso a Paso
9.	API REST con FastAPI
10.	Contenedorización con Docker
11.	Frontend — Interfaz Web
12.	Flujo Completo del Sistema
13.	Problemas Encontrados y Soluciones
14.	Resultados Finales
15.	Referencia de Comandos Clave

1. Descripción del Proyecto

Este proyecto implementa un sistema completo de detección de fatiga en conductores utilizando visión por computadora e inteligencia artificial. El sistema analiza imágenes de la cámara del conductor en tiempo real y clasifica su estado en cuatro categorías: ojo abierto, ojo cerrado, bostezo y no bostezo, generando alertas visuales inmediatas cuando detecta señales de somnolencia.

El sistema consta de tres componentes principales que trabajan de forma integrada:

- **Modelo de IA:** Red neuronal MobileNetV2 con Transfer Learning, entrenada sobre 54,892 imágenes de tres datasets públicos de Kaggle. Accuracy en test: 99.02%.
- **Backend — API REST:** Construida con FastAPI y Python. Recibe fotogramas de la cámara, aplica el pipeline de preprocesamiento (detección de cara con OpenCV Haar Cascade, recorte de región de ojos, CLAHE, normalización) y devuelve la predicción con nivel de alerta.
- **Contenedor Docker:** La API se ejecuta dentro de un contenedor Docker con plataforma linux/amd64 para compatibilidad con Mac M3 (Apple Silicon).
- **Frontend:** Página web con acceso a cámara web en tiempo real. Muestra la clase detectada, porcentaje de confianza, barras de probabilidad para las 4 clases, nivel de alerta con código de color e historial de predicciones recientes.

El flujo completo es: el conductor es capturado por la cámara web → el frontend envía el fotograma a la API cada 800ms → OpenCV detecta la cara y recorta la región de ojos → el modelo MobileNetV2 clasifica el estado → la API responde con la predicción y nivel de alerta → el frontend muestra el resultado en tiempo real.

2. Estructura del Proyecto

El proyecto se organizó con la siguiente estructura de carpetas:

```
Project-fatiga/
■■■ venv/                      # Entorno virtual Python 3.12
■■■ api/
■   ■■■ model/
■   ■   ■■■ modelo_fatiga.keras      # Modelo entrenado (29.9 MB)
■   ■   ■■■ modelo_fatiga_metadata.json  # Metadatos del modelo
■   ■   ■■■ cascades/
■   ■       ■■■ haarcascade_frontalface_default.xml
■   ■       ■■■ haarcascade_eye.xml
■   ■■■ main.py                  # API FastAPI
■   ■■■ requirements.txt         # Dependencias
■   ■■■ Dockerfile              # Configuración Docker
■■■ frontend/
    ■■■ index.html              # Interfaz web con cámara
```

3. Herramientas y Versiones

Herramienta	Versión	Propósito
Python	3.12.13	Lenguaje base del proyecto
TensorFlow / Keras	2.21.0	Entrenamiento y carga del modelo de IA
MobileNetV2	ImageNet	Arquitectura base con Transfer Learning
OpenCV	4.13.0	Detección de cara, recorte de ojos, CLAHE
FastAPI	0.115.0+	Framework para la API REST
Uvicorn	0.30.6+	Servidor ASGI para correr FastAPI
Pillow	latest	Procesamiento de imágenes
NumPy	latest	Operaciones matemáticas con arrays
Docker Desktop	latest	Contenedorización de la API
pyenv	2.6.27	Gestión de versiones de Python
Google Colab T4	GPU	Entrenamiento del modelo en la nube

4. Dataset y Preparación de Datos

4.1 Datasets utilizados

Se utilizaron tres datasets públicos de Kaggle que cubren los distintos estados de fatiga que el sistema debe detectar:

Dataset	Fuente (Kaggle)	Imágenes	Clases
MRL Eye Dataset	prasadvpatil/mrl-dataset	4,000	Open_Eyes, Closed_Eyes
Yawn Eye Dataset	serenaraju/yawn-eye-dataset-new	2,900	yawn, no_yawn, Open, Closed
Drowsiness / CEW	kutaykutlu/drowsiness-detection	47,992	open_eye, closed_eye
TOTAL	—	54,892	4 clases unificadas

4.2 Clases unificadas

Los tres datasets tienen nombres de clases distintos. Se mapearon a un esquema unificado de cuatro clases:

Clase Final	Descripción	Nivel de Alerta
ojo_abierto	Ojo completamente abierto — alerta	NORMAL (verde)
ojo_cerrado	Ojo cerrado — señal de fatiga	PELIGRO (rojo)

Clase Final	Descripción	Nivel de Alerta
bostezo	Boca abierta en bostezo	ATENCIÓN (naranja)
no_bostezo	Boca en posición normal	NORMAL (verde)

4.3 Curación y preprocesamiento

Antes del entrenamiento se aplicó un pipeline de preparación de datos completo documentado en el Entregable 2:

- **Curación:** Verificación de integridad (hash MD5). Solo se eliminaron 8 imágenes corruptas del dataset CEW (0.02%). 54,892 imágenes válidas totales.
- **CLAHE:** Mejora de contraste adaptativa en espacio LAB. MRL Eye mejoró +48% (23.82 → 35.26), Drowsiness/CEW mejoró +60.5% (17.17 → 27.55).
- **Resize:** Todas las imágenes redimensionadas a 224×224 px (estándar MobileNetV2).
- **Normalización:** Valores de píxeles al rango [0, 1].
- **División:** Train 70% (38,419) / Val 15% (8,229) / Test 15% (8,244). Semilla 42.

5. Arquitectura del Modelo de IA

5.1 Arquitectura seleccionada: MobileNetV2 + Transfer Learning

Se seleccionó MobileNetV2 preentrenada en ImageNet como backbone, con una cabeza de clasificación personalizada de cuatro clases. MobileNetV2 fue diseñada para sistemas con restricciones de recursos computacionales, lo que se alinea con el caso de uso real: detección de fatiga en tiempo real desde un dispositivo en el vehículo.

Arquitectura	Parámetros	Top-1 ImageNet	Adecuada para
MobileNetV2 ✓	3.4M	71.8%	Tiempo real, dispositivos ligeros
ResNet50	25.6M	76.0%	Servidores, alta precisión
VGG16	138M	71.3%	Investigación, no producción
EfficientNet-B0	5.3M	77.1%	Balance precisión/tamaño

5.2 Cabeza de clasificación personalizada

```
[Input 224×224×3]
        ↓
[MobileNetV2 backbone — pesos ImageNet]
        ↓
[GlobalAveragePooling2D]
        ↓
[Dense 512, ReLU + BatchNorm + Dropout 0.4]
        ↓
[Dense 256, ReLU + BatchNorm + Dropout 0.3]
        ↓
[Dense 4, Softmax — ojo_abierto | ojo_cerrado | bostezo | no_bostezo]
```

5.3 Estrategia de Transfer Learning en dos fases

- **Fase 1 — Feature Extraction (10 epochs):** Backbone completamente congelado. Solo se entrenó la cabeza de clasificación con LR=1e-3. Val accuracy alcanzada: 97.15%.
- **Fase 2 — Fine-Tuning (15 epochs):** Se descongelaron las últimas 30 capas del backbone con LR=1e-5 (100x menor). Permite especialización en rasgos faciales de fatiga sin destruir los pesos preentrenados. Val accuracy alcanzada: 99.08%.

5.4 Data Augmentation

Se aplicaron augmentaciones durante el entrenamiento para simular condiciones variables de uso real del sistema:

Transformación	Valor	Justificación
Flip horizontal	Sí	Imagen especular del conductor
Rotación aleatoria	±15°	Cámara no siempre perfectamente horizontal
Zoom aleatorio	±10%	Conductor a distintas distancias
Desplazamiento	±10%	Cara no siempre centrada en el encuadre
Variación de brillo	80%–120%	Luz solar, noche, túneles, distintas horas
Flip vertical	No	Una cara boca abajo no tiene sentido

6. Entrenamiento y Resultados

6.1 Configuración de entrenamiento

Parámetro	Fase 1	Fase 2
Backbone	Congelado	Últimas 30 capas libres
Learning Rate	1e-3	1e-5
Epochs máximos	10	15
Optimizer	Adam	Adam
Loss	Sparse Categorical CE	Sparse Categorical CE
Batch size	32	32
EarlyStopping	patience=5	patience=6
ReduceLROnPlateau	factor=0.5, patience=3	factor=0.5, patience=3

6.2 Resultados del entrenamiento

Métrica	Fase 1	Fase 2
Val Accuracy	97.15%	99.08%
Test Accuracy	—	99.02%
Parámetros totales	3,049,284	3,049,284
Tamaño del modelo	—	29.9 MB
Plataforma	Colab T4 GPU	Colab T4 GPU

6.3 Métricas por clase — Conjunto Test

El modelo alcanzó un rendimiento sobresaliente en las cuatro clases. El recall en ojo_cerrado y bostezo es especialmente relevante para un sistema de seguridad vial: se priorizó reducir los falsos negativos (no detectar fatiga cuando existe) sobre reducir los falsos positivos (alarmas innecesarias).

Clase	Precision	Recall	F1-Score	Importancia
ojo_abierto	~0.99	~0.99	~0.99	Base — estado normal
ojo_cerrado	~0.99	~0.99	~0.99	CRÍTICA — señal de fatiga
bostezo	~0.98	~0.98	~0.98	ALTA — señal de somnolencia

Clase	Precision	Recall	F1-Score	Importancia
no_bostezo	~0.99	~0.99	~0.99	Base — estado normal
Accuracy total	—	—	99.02%	—

7. Pipeline de Preprocesamiento en Producción

Una diferencia clave entre los datasets de entrenamiento y el uso en producción es que los datasets (MRL Eye y CEW) contienen imágenes de ojos recortados, mientras que la cámara web captura el rostro completo del conductor. Para resolver esto se implementó un pipeline de preprocesamiento inteligente en la API:

```
1. Decodificar imagen desde bytes (PIL → NumPy → BGR)
2. Detectar cara con Haar Cascade (haarcascade_frontalface_default.xml)
3a. SI hay cara detectada:
    → Recortar mitad superior (región de ojos)
    → Detectar ojos individuales con haarcascade_eye.xml
    → Recortar el ojo con mayor área detectada
    → Aplicar pipeline: CLAHE → Resize 224×224 → Normalización [0,1]
    → Modo: "ojos_recortados"
3b. SI no hay cara (imagen estática, mal encuadre):
    → Usar imagen completa → CLAHE → Resize → Normalización
    → Modo: "imagen_directa"
4. Predicción con modelo MobileNetV2
5. Retornar clase, confianza, nivel de alerta, modo y tiempo de inferencia
```

La respuesta de la API incluye el campo **modo_preprocesamiento** que indica cuál de las rutas tomó el pipeline, y **cara_detectada** (booleano) para transparencia sobre la calidad de la detección.

8. Proceso de Implementación — Paso a Paso

8.1 Entrenamiento en Google Colab (GPU T4)

El modelo fue entrenado en Google Colab con GPU T4 usando Python 3.10 y TensorFlow 2.21.0. El notebook del Entregable 3 cubre todo el pipeline: descarga de datasets desde Kaggle, curación, preprocesamiento CLAHE, construcción de la arquitectura MobileNetV2, Data Augmentation con tf.data, entrenamiento en dos fases y exportación del modelo.

- Descarga de los 3 datasets desde Kaggle con autenticación kaggle.json
- Pipeline de curación: verificación MD5, eliminación de corruptas y duplicados
- Unificación de clases y división estratificada 70/15/15 con sklearn
- Construcción de MobileNetV2 + cabeza Dense 512→256→4
- Entrenamiento Fase 1: 10 epochs, backbone congelado, LR=1e-3
- Entrenamiento Fase 2: 15 epochs, fine-tuning últimas 30 capas, LR=1e-5
- Descarga del modelo: modelo_fatiga.keras (29.9 MB) y modelo_fatiga_metadata.json

8.2 Instalación de Python 3.12 con pyenv

El modelo fue entrenado en Colab con Python 3.10/3.12. Para garantizar compatibilidad local en Mac M3, se usó pyenv para instalar Python 3.12.13:

```
brew update && brew upgrade pyenv
pyenv install 3.12.13
eval "$(pyenv init -)"
pyenv local 3.12.13
python --version # Python 3.12.13
```

8.3 Creación de la estructura de carpetas

```
cd ~/Documents
mkdir Project-fatiga
cd Project-fatiga
mkdir -p api/model
mkdir frontend
```

Los archivos modelo_fatiga.keras y modelo_fatiga_metadata.json se colocaron en api/model/. Los archivos main.py, requirements.txt y Dockerfile en api/. El frontend/index.html en frontend/.

8.4 Creación del entorno virtual

```
cd ~/Documents/Project-fatiga
python -m venv venv
source venv/bin/activate
```

Nota: En Mac M3 con Anaconda instalado, el prompt muestra ((venv)) (base). Esto es normal. Es importante activar el venv antes de correr uvicorn, de lo contrario el sistema usará el Python de Anaconda que no tiene las dependencias.

8.5 Instalación de dependencias

tensorflow-macos ya no está disponible para versiones recientes. Se instaló tensorflow directamente, que en Mac M3 instala la versión ARM64:

```
pip install fastapi uvicorn tensorflow numpy pillow opencv-python python-multipart
pydantic
```

Versiones instaladas: TensorFlow 2.21.0, FastAPI 0.136.3, OpenCV 4.13.0.

8.6 Prueba local con Uvicorn

```
cd ~/Documents/Project-fatiga
source venv/bin/activate
cd api
uvicorn main:app --reload
```

Al arrancar por primera vez, la API descarga automáticamente los archivos Haar Cascade de OpenCV desde el repositorio oficial. Se verificó en:

- http://127.0.0.1:8000 — Health check con info del modelo
- http://127.0.0.1:8000/docs — Swagger UI automático de FastAPI
- http://127.0.0.1:8000/modelo/info — Detalles del modelo y preprocesamiento

9. API REST con FastAPI

9.1 Endpoints

Método	Endpoint	Descripción
GET	/	Health check. Retorna estado del modelo, versión y clases.
POST	/predict	Recibe imagen, aplica preprocesamiento inteligente y retorna predicción.
GET	/modelo/info	Información detallada del modelo, arquitectura y métricas.

9.2 Respuesta del endpoint /predict

```
{
  "clase": "ojo_cerrado",
  "probabilidad": 0.9923,
  "nivel_alerta": "PELIGRO",
  "color_alerta": "#F44336",
  "mensaje_alerta": "Ojos cerrados — posible fatiga",
  "todas_las_clases": {
    "ojo_abierto": 0.0041,
    "ojo_cerrado": 0.9923,
    "bostezo": 0.0022,
    "no_bostezo": 0.0014
  },
  "tiempo_inferencia_ms": 43.7,
  "modo_preprocesamiento": "ojos_recortados",
  "cara_detectada": true
}
```

9.3 Niveles de alerta

Clase detectada	Nivel	Color	Acción sugerida
ojo_abierto	NORMAL	Verde #4CAF50	Sin acción
no_bostezo	NORMAL	Verde #4CAF50	Sin acción
bostezo	ATENCIÓN	Naranja #FF9800	Sugerir descanso
ojo_cerrado	PELIGRO	Rojo #F44336	Alarma inmediata

10. Contenedorización con Docker

10.1 Dockerfile

La diferencia crítica respecto a otros proyectos es la línea FROM con la plataforma explícita, necesaria para Mac M3 (Apple Silicon):

```
FROM --platform=linux/amd64 python:3.12-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    gcc libglib2.0-0 libsm6 libxext6 libxrender-dev libgl1-mesa-glx

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

10.2 Build y ejecución

```
# Build (desde la carpeta api/)
docker buildx build --platform linux/amd64 -t img-api-fatiga .

# Correr contenedor
docker run -d --name srv-api-fatiga -p 8000:8000 img-api-fatiga:latest

# Verificar
docker ps
```

11. Frontend — Interfaz Web

Se creó el archivo frontend/index.html con acceso a la cámara web del dispositivo y visualización en tiempo real de los resultados del modelo:

- **Acceso a cámara:** HTML5 getUserMedia con resolución 640×480, modo usuario (frontal).
- **Predicción automática:** Captura un fotograma cada 800ms y lo envía a la API sin necesidad de presionar botón.
- **Panel de resultado:** Muestra la clase detectada en grande con nivel de alerta codificado por color (verde/naranja/rojo).
- **Barras de probabilidad:** Visualización animada de la confianza del modelo para las cuatro clases simultáneamente.
- **Modo de preprocesamiento:** Indica si detectó cara y qué pipeline usó (ojos_recortados / cara_completa / imagen_directa).
- **Historial:** Muestra las últimas 12 predicciones de la sesión.
- **Subida de imagen:** Permite analizar imágenes estáticas además de la cámara.
- **Estado de API:** Verifica la conexión con la API cada 30 segundos.

12. Flujo Completo del Sistema

```
[ Cámara web del conductor ]
        |  Fotograma capturado cada 800ms (JPEG)
        ↓
[ Frontend HTML5 — fetch POST /predict ]
        |  multipart/form-data
        ↓
[ Docker Puerto 8000:8000 ]
        |
        ↓
[ FastAPI + Uvicorn (main.py) ]
        |  Decodificar imagen
        ↓
[ OpenCV Haar Cascade — detectar cara ]
        |  Recortar región de ojos
        ↓
[ Pipeline: CLAHE → Resize 224×224 → Normalización ]
        |
        ↓
[ Modelo MobileNetV2 (modelo_fatiga.keras) ]
        |  4 probabilidades (softmax)
        ↓
[ JSON Response: clase + confianza + nivel_alerta + modo ]
        |
        ↓
[ Frontend: alerta visual + barras + historial ]
```

13. Problemas Encontrados y Soluciones

Problema	Causa	Solución
tensorflow-macos no encontrado	Ya no existe para versiones recientes de TF	Usar tensorflow sin sufijo; en M3 instala ARM64 automáticamente
ModuleNotFoundError: cv2	Uvicorn usando Python de Anaconda en lugar del venv	Siempre activar el venv antes: source venv/bin/activate
Ojos abiertos/cerrados no detectados bien	Datasets de entrenamiento tienen imágenes muy diferentes a la cámara web	Pipeline con CLAHE, cámara frontal, Haar Cascade completa para, recorta región de o
Docker build falla en Mac M3	Plataforma ARM no compatible con tensorflow-cpu	Agregar --platform linux/amd64 en FROM del Dockerfile
Bajo contraste en MRL Eye	Imágenes originales con std ~17-24 (muy bajo)	CLAHE en espacio LAB: +48% y +60.5% de mejora respectivament
Clases duplicadas en Yawn_Eye	Dataset tiene subcarpetas anidadas con nombres iguales	drop_duplicates por ruta en el inventario unificado

14. Resultados Finales

Componente	Estado	Detalle
Dataset unificado	■ Listo	54,892 imágenes, 4 clases, split 70/15/15
Pipeline preprocesamiento	■ Validado	CLAHE +48-60%, resize 224×224, normalización
Modelo MobileNetV2	■ Entrenado	99.02% test accuracy, 29.9 MB
Transfer Learning Fase 1	■ Completo	97.15% val accuracy
Transfer Learning Fase 2	■ Completo	99.08% val accuracy (fine-tuning)
API FastAPI local	■ Probada	uvicorn main:app --reload
Haar Cascade detección cara	■ Activo	Descarga automática al iniciar
Pipeline inteligente ojos	■ Funcionando	Recorte región ojos mejora detección
modelo_fatiga.keras	■ Descargado	Listo para Docker y producción
Docker imagen	■ Pendiente	img-api-fatiga con --platform linux/amd64
Frontend cámara web	■ Funcionando	Predicción en tiempo real cada 800ms
Swagger UI	■ Disponible	http://localhost:8000/docs

15. Referencia de Comandos Clave

Propósito	Comando
Instalar Python 3.12	pyenv install 3.12.13
Activar versión pyenv	eval "$(pyenv init -)" && pyenv local 3.12.13
Crear entorno virtual	python -m venv venv
Activar entorno virtual	source venv/bin/activate
Instalar dependencias	pip install fastapi uvicorn tensorflow numpy pillow opencv-python python-multipart pyd
Correr API local	uvicorn main:app --reload
Build Docker Mac M3	docker buildx build --platform linux/amd64 -t img-api-fatiga .
Correr contenedor	docker run -d --name srv-api-fatiga -p 8000:8000 img-api-fatiga:latest
Ver contenedores activos	docker ps
Detener contenedor	docker stop srv-api-fatiga
Eliminar contenedor	docker rm srv-api-fatiga

Propósito	Comando
Ver logs del contenedor	docker logs srv-api-fatiga
Health check API	curl http://localhost:8000/